SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. (L2)

Assessment Tool : Data Interpretation

Weightage: 20%

QUESTIONS: **Total Marks: 20**

1: HTTP Response Analysis

A web server returns the following HTTP response header:

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive
```

Questions:

1. Identify any incorrect or invalid fields in the header.
2. Analyse the issue with `Content-Length` and explain its impact.
3. Determine whether the status code matches the response correctness.
4. Suggest corrections for the identified errors.

ANSWER 1: HTTP Response Analysis

a) Incorrect or invalid fields in the header

Two header fields are wrong, and one is incomplete:

Field	Value given	Status	Reason
Content-Length	-150	Invalid	Must be a non-negative integer (a count of bytes in the body). A negative size is impossible.
Date	Mon, 01 Apr 2026 10:00:00 GMT	Inconsistent	1 April 2026 falls on a Wednesday, not a Monday. The day name and date do not match, so the value is not a correct HTTP-date.
Content-Type	text/html	Incomplete	Valid, but no character set is declared. Best practice is text/html; charset=UTF-8.
Connection	keep-alive	Valid	Allowed (it is already the default in HTTP/1.1), so it is only redundant.
Status line	HTTP/1.1 200 OK	Valid syntax	Correct format, but see part (c) for whether it fits the response.

b) Issue with Content-Length and its impact

Content-Length tells the client exactly how many bytes of message body to read. A value of -150 is meaningless, so the client cannot find where the body ends.

- **Parsing failure:** Per the HTTP/1.1 specification (RFC 7230), a recipient that finds an invalid Content-Length must treat the message as an error and discard it.
- **Page may not load:** The browser may show an error or a blank/partial page instead of the HTML.
- **Connection problems:** With keep-alive, the client cannot tell where one response ends and the next begins, so the persistent connection must be closed. This hurts performance.
- **Security risk:** Disagreement over message length between a proxy and a server can lead to response splitting / request smuggling attacks.

c) Does the status code match the response correctness?

No. The status code 200 OK says the request succeeded and the response is valid. But the response is malformed (invalid Content-Length and mismatched Date), so what the status claims does not match what is actually delivered. The code itself is syntactically correct; it is the message framing that contradicts it.

- If the fault is on the server side, fix the headers and keep 200 OK.
- If the server cannot build a correct response, it should return 500 Internal Server Error.
- If the fault came from a malformed client request, the proper code is 400 Bad Request.

d) Suggested corrections

```
HTTP/1.1 200 OK
Date: Wed, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html; charset=UTF-8
Content-Length: 150
Connection: keep-alive
```

- Set **Content-Length** to the real size of the body in bytes (150 is used here only as an example value).
- Correct the **Date** to Wed (or change the date to a real Monday). Servers should generate it automatically from the system clock.
- Add **charset=UTF-8** to Content-Type.
- If the body size is not known in advance, use **Transfer-Encoding: chunked** instead. Never send both headers together.

> **Conclusion:** The response header is not valid. The negative Content-Length is the critical fault because it breaks message framing, and the wrong weekday in the Date header is a secondary fault. A 200 status cannot be trusted until both are fixed.

2: DOM Structure Analysis

Consider the following DOM tree representation:

```
<html>
  <head>
    <title>Test Page</title>
  </head>
  <body>
    <div>
      <p>Hello World
    </div>
  </body>
</html>
```

Questions:

1. Identify missing or improperly closed elements.
2. Analyze how the browser will interpret this structure.
3. Determine the impact on rendering and layout.
4. Suggest corrections to make the DOM valid.

ANSWER 2: DOM Structure Analysis

a) Missing or improperly closed elements

- The ** ** element is opened with "Hello World" but is **never closed** (the end tag is missing).
- All other elements (html, head, title, body, div) are correctly opened and closed.
- Additional (non-critical) omissions: no <!DOCTYPE html>, no lang attribute on <html>, and no <meta charset> in <head>.

b) How the browser will interpret this structure

HTML parsers are forgiving. When the browser reaches the end tag while a is still open, its error-recovery rules automatically close the first and then close the . The tree the browser builds in memory is:

```
html
 ├─ head
 │    └─ title  →  "Test Page"
 └─ body
      └─ div
           └─ p  →  "Hello World"   (closed automatically)
```

So the browser silently repairs the mistake and produces the same DOM as if the tag had been written correctly. A parse error is still recorded, which developer tools or a validator can report.

c) Impact on rendering and layout

- **In a normal HTML page:** Almost no visible impact. "Hello World" appears as a paragraph block inside the div, with the default top and bottom paragraph margins.
- **If more content followed:** Some tags (for example another or) would auto-close the paragraph, but inline content could be wrongly absorbed into it, causing unexpected spacing or styling.
- **In XHTML/XML mode:** An XML parser is strict. The unclosed is a well-formedness error, so the page would not render and an error message would be shown instead.
- **Other effects:** CSS selectors and JavaScript DOM code may behave unpredictably on larger pages, and validators will report the page as invalid.

d) Corrections to make the DOM valid

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <title>Test Page</title>
  </head>
  <body>
    <div>
      <p>Hello World</p>
    </div>
  </body>
</html>
```

> **Conclusion:** The only real defect is the missing . Browsers hide the error through automatic repair, but writing well-formed markup is necessary for standards compliance, XHTML/XML use, and predictable layout.

3: GET vs POST in Login System

A login system uses two different methods:

Method	URL Example	Data Visibility	Security Level
GET	/login?user=admin&pass=1234	Visible in URL	Low
POST	/login	Hidden in body	Moderate

Questions:

1. Interpret the differences in data transmission between GET and POST.
2. Analyze which method is more suitable for login and why.
3. Identify security risks in using GET for authentication.
4. Suggest best practices for secure login implementation.

ANSWER 3: GET vs POST in Login System

a) Differences in data transmission

Aspect	GET	POST
Where data travels	In the URL query string (/login?user=admin&pass=1234)	In the request body, not in the URL
Visibility	Visible in address bar, browser history, bookmarks, server and proxy logs, Referer header	Not shown in the URL or history (but still readable on the network unless HTTPS is used)
Size limit	Limited by URL length (about 2,000 characters in practice)	Practically unlimited
Caching / bookmarking	Can be cached and bookmarked	Not cached or bookmarked by default
Purpose	Retrieve data (safe, repeatable)	Submit data / change server state
Security level	Low	Moderate (high only when combined with HTTPS)

b) Which method is more suitable for login, and why

POST is the suitable method. Login submits sensitive credentials and creates a session, which is a state-changing action. POST keeps the username and password out of the URL, so they are not saved in browser history, bookmarks, or server access logs, and are not leaked through the Referer header. POST also matches the intended meaning of the HTTP methods (GET reads, POST submits).

Note that POST alone is not encryption. Credentials are still sent as readable text unless the connection uses HTTPS.

c) Security risks of using GET for authentication

- **Credential exposure in the URL:** The password appears on screen and can be seen by anyone nearby (shoulder surfing).
- **Stored in browser history and bookmarks:** Anyone using the same device can retrieve the password.
- **Logged by servers and proxies:** Web server, proxy and CDN logs record full URLs, storing passwords in plain text.
- **Leaked through the Referer header:** Clicking a link to another site can send the login URL to that site.
- **Cached and shareable:** The URL can be cached or copied and shared, taking the credentials with it.
- **Easy to attack:** Login URLs can be triggered by an image tag or link (CSRF-style attacks) and are simple to brute-force or replay.

d) Best practices for secure login implementation

- Send credentials with **POST over HTTPS (TLS)** only, and enable **HSTS** so browsers never use plain HTTP.
- **Hash passwords** with a slow, salted algorithm such as bcrypt, scrypt or Argon2. Never store or log plain-text passwords.

- Add **CSRF tokens** to login forms.
- Use **rate limiting, account lockout or CAPTCHA** to stop brute-force and credential-stuffing attacks.
- Offer **multi-factor authentication (MFA)**.
- Set session cookies with **HttpOnly, Secure and SameSite** flags, and regenerate the session ID after login.
- Show **generic error messages** ("Invalid username or password") so attackers cannot tell which one was wrong.
- Validate and sanitise input, and use **parameterised queries** to prevent SQL injection.
- Send **Cache-Control: no-store** on login pages and responses.

> **Conclusion:** POST over HTTPS is the correct choice for login. GET must never carry credentials because the URL is stored and exposed in many places. Real security comes from combining POST with encryption, password hashing, and session protection.

4: HTML Code Inspection

Analyze the following HTML snippet:

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph
<img src="image.jpg">
</body>
</html>
```

Questions:

1. Identify missing closing tags.
2. Analyze issues with the `<img>` tag.
3. Determine how browsers will handle these errors.
4. Suggest a corrected version of the code.

ANSWER 4: HTML Code Inspection

a) Missing closing tags

- **</h1>** is missing after "Welcome".
- **</p>** is missing after "This is a paragraph".
- <img> needs no closing tag. It is a void (empty) element in HTML5. In XHTML it would have to be self-closed as .

b) Issues with the <img> tag

- **No alt attribute:** This is required by the HTML standard. Without it, screen-reader users get no description, search engines cannot understand the image, and nothing is shown if the image fails to load.
- **No width and height:** The browser cannot reserve space before the image loads, so the layout can jump (layout shift).

- **Unverified source:** "image.jpg" is a relative path and the image must actually exist in the same folder, otherwise a broken-image icon appears.
- **Placement:** The tag sits inside the unclosed paragraph, so it becomes part of that structure (see part c).

c) How browsers will handle these errors

The browser does not stop. It repairs the markup using the HTML5 parsing rules:

- The start tag does **not** close an open <h1>, so the paragraph becomes a **child of the heading**.
- The <img> then becomes a child of that paragraph.
- At </body>, the browser automatically closes the open and <h1>.

The resulting tree is:

```
body
 └─ h1
      ├─ "Welcome"
      └─ p
           ├─ "This is a paragraph"
           └─ img
```

Effect: The paragraph text inherits the large, bold heading style, and the image sits inside the heading. Screen readers announce the whole content as a heading, which hurts accessibility and SEO. The page displays, but not as intended.

d) Corrected version of the code

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Sample</title>
</head>
<body>
  <h1>Welcome</h1>
  <p>This is a paragraph</p>
  <img src="image.jpg" alt="Sample image" width="300" height="200">
</body>
</html>
```

> **Conclusion:** The missing </h1> and cause the paragraph and image to be nested inside the heading, producing wrong styling and poor accessibility. Closing the tags properly and adding alt, width and height produces valid, accessible and predictable HTML.

5: Browser-Server Communication Flow

The following sequence describes a web request:

1. User enters a URL in the browser
2. DNS lookup occurs
3. HTTP request is sent
4. Server processes request
5. Response is returned
6. Browser renders page

Questions:

1. Interpret each step in the communication flow.
2. Identify where delays can occur.
3. Analyze the role of DNS in the process.
4. Suggest optimizations to improve performance.

ANSWER 5: Browser-Server Communication Flow

a) Interpretation of each step

Step	What happens
1. User enters a URL	The browser splits the URL into protocol (http/https), domain, port and path. It checks its cache and HSTS list before going to the network.
2. DNS lookup occurs	The domain name is converted to an IP address. The browser cache, OS cache and hosts file are checked first, then a recursive DNS resolver queries the root, TLD and authoritative name servers.
3. HTTP request is sent	The browser opens a TCP connection to the IP (three-way handshake) and, for HTTPS, completes a TLS handshake. It then sends a request such as GET /index.html HTTP/1.1 with headers like Host, User-Agent and Cookie.
4. Server processes request	The web server routes the request, runs application code, may query a database or read files, and builds the response.
5. Response is returned	The server sends a status line (e.g. 200 OK), headers (Content-Type, Content-Length, caching) and the body (HTML). Redirects such as 301/302 trigger a new request.
6. Browser renders page	The browser parses HTML into the DOM and CSS into the CSSOM, combines them into a render tree, then does layout and paint. Linked CSS, JS, images and fonts cause further requests.

b) Where delays can occur

Stage	Possible cause of delay
DNS lookup	Uncached lookups, slow or distant resolver, many domains to resolve
Connection setup	TCP and TLS handshakes cost multiple round trips, especially over long distances
Network transfer	High latency, low bandwidth, packet loss, large uncompressed files
Server processing	Slow database queries, heavy computation, overloaded or under-powered server
Redirects	Each redirect adds a full extra request cycle
Rendering	Render-blocking CSS/JS, large images, too many sub-resource requests, heavy JavaScript

c) Role of DNS in the process

DNS (Domain Name System) works like the internet's phone book. Computers communicate using IP addresses, but people use names such as example.com. DNS translates the name

into the IP address of the server, so without it the browser would not know where to send the HTTP request.

- It is **hierarchical**: root servers, then TLD servers (.com), then the domain's authoritative server.
- Answers are **cached** according to a TTL (time-to-live), so repeat visits skip the lookup.
- It supports **load balancing and CDN routing** by returning the nearest or least-busy server.
- It is a **delay point and an attack target** (DNS spoofing/poisoning); DNSSEC adds protection.

d) Optimizations to improve performance

- **DNS:** Use a fast DNS provider, set sensible TTLs, and use dns-prefetch and preconnect hints.
- **Connections:** Use HTTP/2 or HTTP/3, TLS 1.3, and keep-alive connections to reduce handshakes.
- **Content delivery:** Serve static files from a CDN close to the user.
- **Caching:** Set Cache-Control / ETag headers so the browser reuses files; add server-side caching (e.g. Redis).
- **Smaller payloads:** Enable gzip/Brotli compression, minify HTML/CSS/JS, and compress images (WebP/AVIF).
- **Server:** Optimize code, index database queries, and scale or load-balance under heavy traffic.
- **Rendering:** Load scripts with defer/async, inline critical CSS, lazy-load images, and avoid unnecessary redirects.

> **Conclusion:** A web page load is a chain of dependent steps: DNS, connection, request, server processing, response and rendering. A delay in any step slows the whole page. DNS is the essential first link, and combining DNS, network, caching, compression and rendering optimizations gives the biggest speed improvement.

Code of Conduct:

I **K. Naga Venkata Siva / 192425345** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:
K. Naga Venkata Siva
192425345

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand